Stochastic Gradient Descent

Jyotishka Datta (jyotishka@vt.edu)
Department of Statistics, Virginia Tech

Wiki Definition

Stochastic gradient descent (often shortened to SGD), also known as incremental gradient descent, is a stochastic approximation of the gradient descent optimization and iterative method for minimizing an objective function that is written as a sum of differentiable functions.

In other words, SGD tries to find minima or maxima by iteration.

Stochastic gradient descent

To illustrate the use of stochastic gradient descent, let us try to use it to solve for the ordinary least squares solution in linear regression. I will first generate a small test set of data:

```
set.seed(42)
n <- 1000
p <- 10
X <- matrix(rnorm(n*p),ncol=p)
beta <- 1 / 1:p
y <- X %*% beta + rnorm(n, sd=0.5)
```

True Solution

The true solution, which in this case we can calculate analytically, is given by directly solving the normal equations. We can get these in R by the following:

```
betaOls <- coef(lm(y ~ X - 1))
```

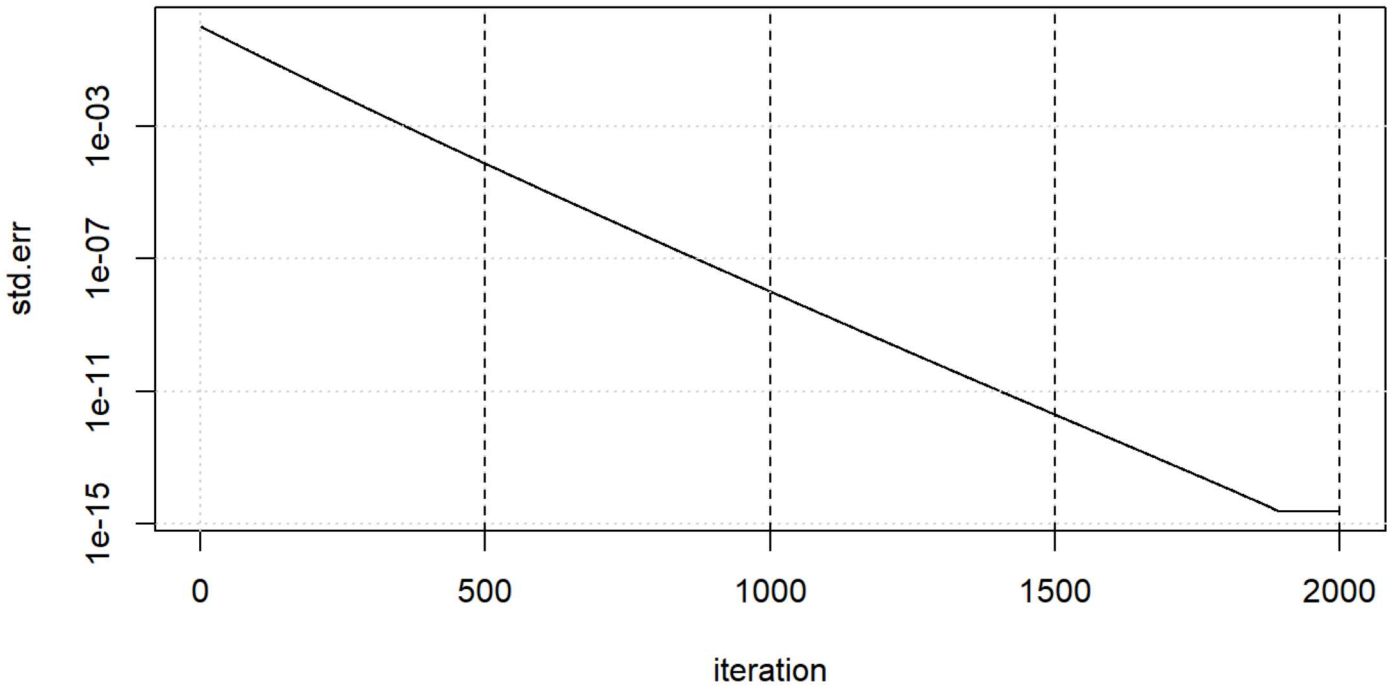
Standard Gradient Descent

Standard gradient descent calculates the entire gradient function and moves some amount in the opposite direction of the gradient. Here we run this algorithm for 2000 iterations and plot the error on the log scale:

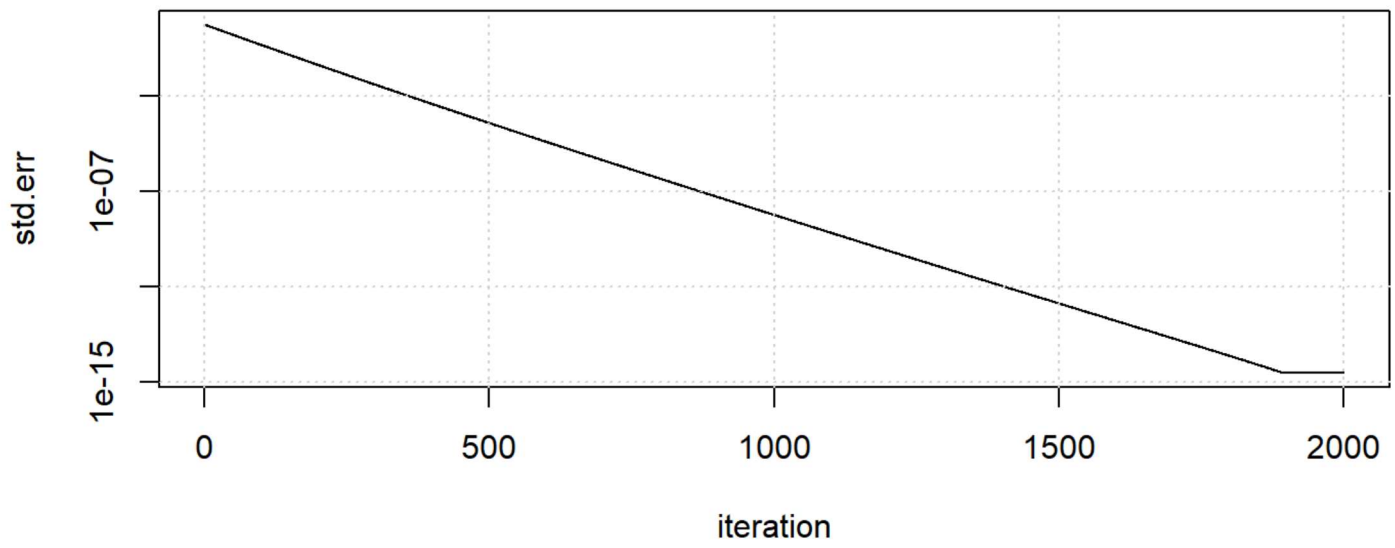
```
eta <- 0.01
b <- rep(0, p)
E <- 2000
std.err <- rep(NA, E)
for (k in 1:E) {
  gradFull <- (2/n) * ((t(X) %*% X) %*% b - t(X) %*% y)
  b <- b - eta * gradFull
  std.err[k] <- max(abs(b - betaOls))
}
```

Standard Gradient Descent (Contd.)

```
plot(std.err, log="y", type="l", xlab="iteration"); grid()
abline(v=500*(1:4), lty = 2)
```



Standard Gradient Descent (Contd.)



Notice that the log error rate is decreasing almost exactly as a linear function of the iteration.

Mini-batch

I now construct a matrix of mini-batch samples. For example, with a size of 5 I get the following:

```
m <- 5
miniBatchIndex <- matrix(sample(1:n),ncol=m)
head(miniBatchIndex)
```

```
##      [,1] [,2] [,3] [,4] [,5]
## [1,]  385  961  208  758  589
## [2,]  381  248  101  611  264
## [3,]  678   69  807  681  411
## [4,]  754  642  757  214  725
## [5,]   62  974  293  177  586
## [6,]  893  865  392  738  474
```

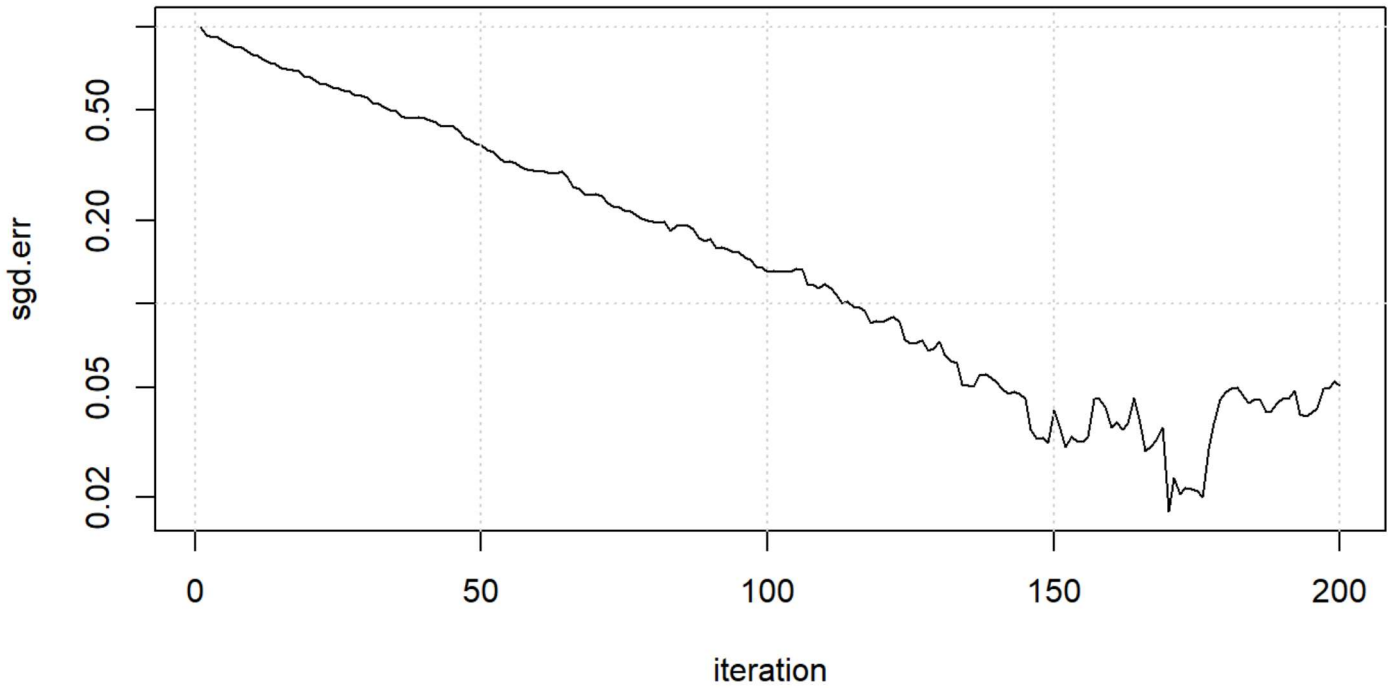
Mini-batch Gradient

Using these minibatch estimates, I want to calculate an **estimate of the gradient function** and again move a small direction downhill from the current value of beta.

```
eta <- 0.01
b <- rep(0, p)
sgd.err <- rep(NA, nrow(miniBatchIndex))
for (j in 1:nrow(miniBatchIndex)) {
  ind <- miniBatchIndex[j,]
  gradMini <- (2/m) * ((t(X[ind,]) %*% X[ind,]) %*% b - t(X[ind,]) %*% y[ind])
  b <- b - eta * gradMini
  sgd.err[j] <- max(abs(b - betaOls))
}
```

Mini-batch Gradient Descent

```
plot(sgd.err, log="y", type="l", xlab="iteration")
grid()
```



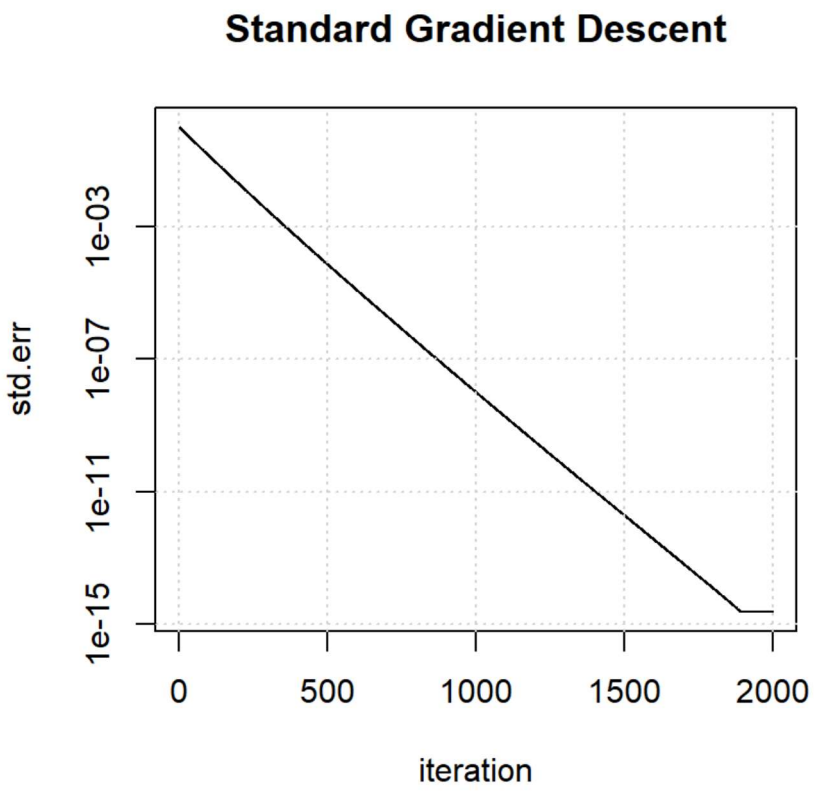
Mini-batch Gradient Descent

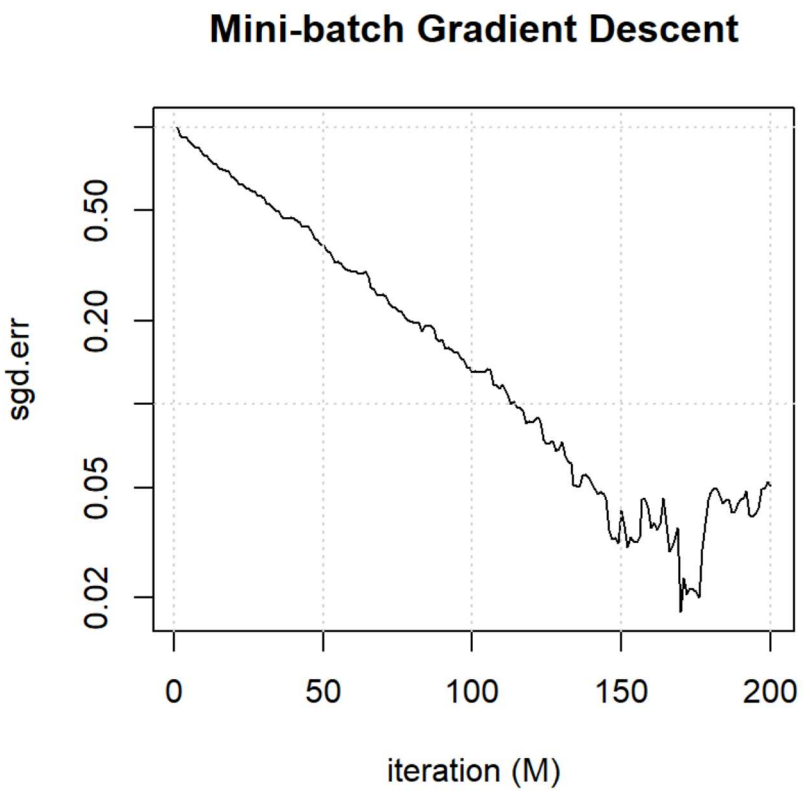
Notice that this curve is much more noisy, but gets under an error rate of 0.05 in **only one pass through the data**.

Be careful to see that the **200 iterations here is the equivalent of one iteration in the gradient descent approach**.

Pause for a minute !!

Difference





Each point is one complete pass through the data.

This is just one pass through the data!

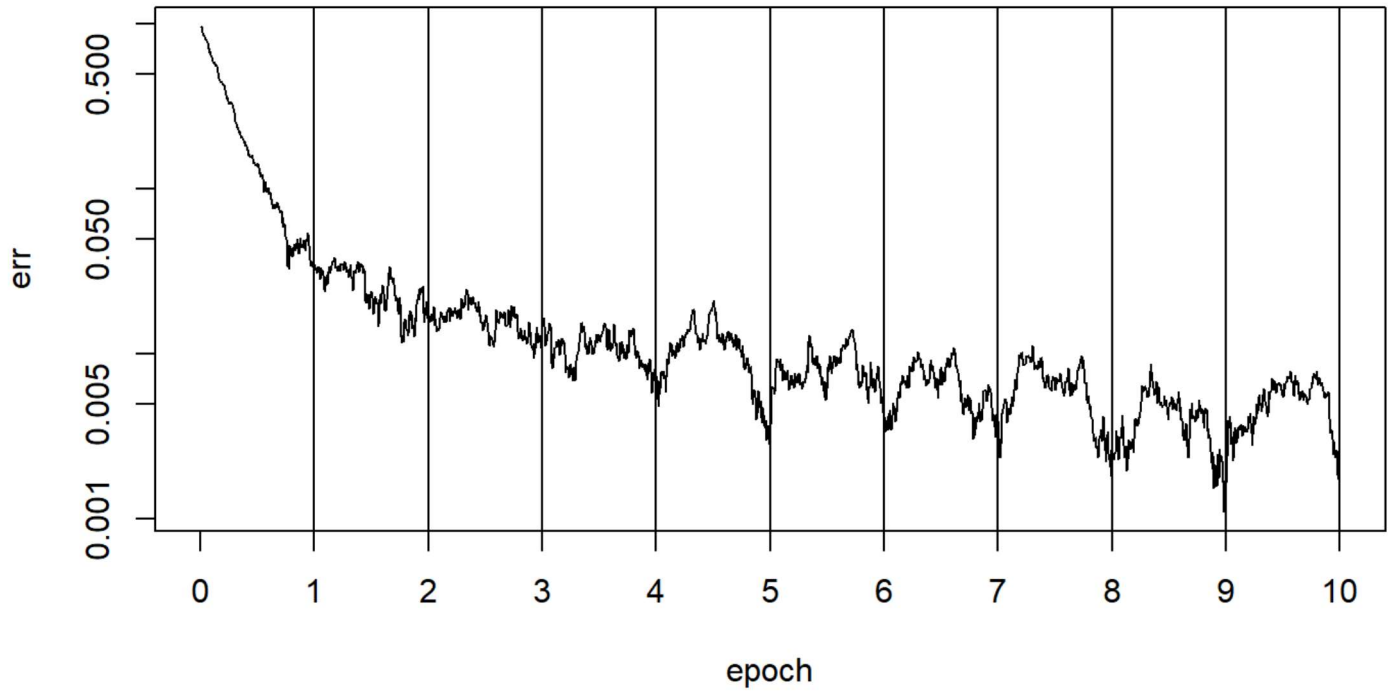
Increasing Epoch

Tweaking this code slightly, let's iterate over 10 epochs and see how stochastic gradient descent converges over a longer number of cycles.

```
eta <- 0.01
b <- rep(0, p)
E <- 10
err <- rep(NA, nrow(miniBatchIndex)*E)
for (k in 1:E) {
  miniBatchIndex <- matrix(sample(1:n),ncol=m)
  for (j in 1:nrow(miniBatchIndex)) {
    ind <- miniBatchIndex[j,]
    gradMini <- (1/m) * (2 * (t(X[ind,]) %*% X[ind,]) %*% b - 2 * t(X[ind,]) %*% y[ind])
    b <- b - eta * gradMini
    err[j+(k-1)*nrow(miniBatchIndex)] <- max(abs(b - betaOls))
  }
}
```

SGD with E = 10

```
plot(err, log="y", type="l", axes=FALSE, xlab="epoch")
box(); axis(2)
axis(1, at=(0:E)*nrow(miniBatchIndex), 0:E)
abline(v=(1:E)*nrow(miniBatchIndex))
```



SGD with E = 10

Notice that this does not actually appear to converge, but rather is essentially a random walk after the first epoch.

The issue is that we need an adaptive learning rate.

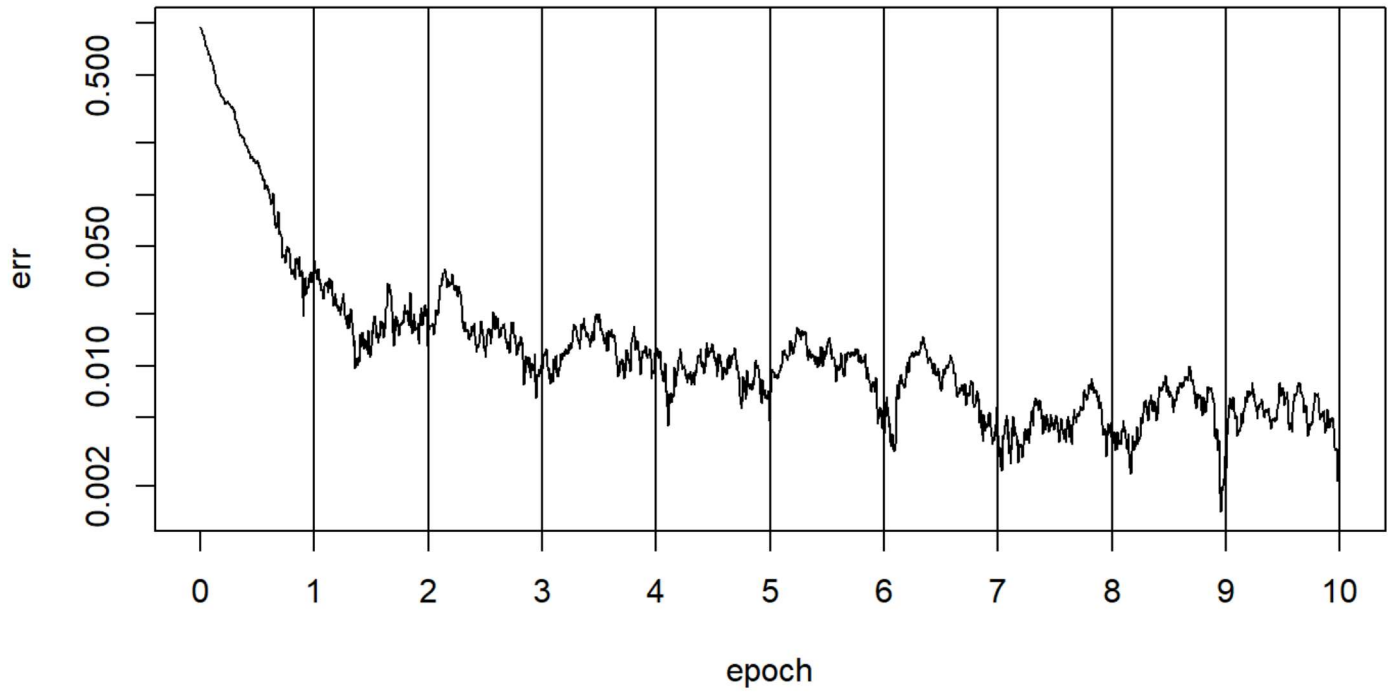
Adaptive Learning rate

Here we divide the **original learning rate** η by the epoch number, this causes the movements to slow down over time.

```
eta <- 0.01
b <- rep(0, p)
m <- 5
E <- 10
err <- rep(NA, nrow(miniBatchIndex)*E)
for (k in 1:E) {
  miniBatchIndex <- matrix(sample(1:n),ncol=m)
  for (j in 1:nrow(miniBatchIndex)) {
    ind <- miniBatchIndex[j,]
    gradMini <- (1/m) * (2 * (t(X[ind,]) %*% X[ind,]) %*% b - 2 * t(X[ind,]) %*% y[ind])
    b <- b - (eta) * (1 / k) * gradMini
    err[j+(k-1)*nrow(miniBatchIndex)] <- max(abs(b - betaOls))
  }
}
```

Adaptive Learning Rate

```
plot(err, log="y", type="l", axes=FALSE, xlab="epoch")
box(); axis(2)
axis(1, at=(0:E)*nrow(miniBatchIndex), 0:E)
abline(v=(1:E)*nrow(miniBatchIndex))
```



Adaptive Learning Rate

There is still a good amount of noise in the middle of the epoch (you can think of this as a type of bias given that some data points have been used more than others),

… but at the end of each epoch the value has improved compared to the value at the end of the prior epoch.

We are able to get to within 0.001 in just 10 cycles of the data, something which would takes **nearly 500 iterations** in the non-stochastic variant.